

EE-222 Data Structures and Algorithms

BS(CE)-2k21 LAB

REPORT # 7



Submitted By:

Reg. No.	Name
21-CE-009	Huzaifa Shahbaz

Department of Computer Engineering

HITEC University Taxila

	Presentation (Format(0.5)+ Title (0.5)+ Conclusion(1) +Procedure(2) +Objective(1))	Calculation/ Coding	Observation/ Program Results	Total
Total	5	5	5	15
Obtained				

Lab Instructor.
Kaynat Rana

Experiment #7

Infix, Prefix and Postfix

Objective:

The objective of this lab is to understand the conversions between infix to prefix and postfix

- Infix to prefix
- Infix to postfix
- Prefix to postfix and vice versa

Equipment Used:

Microsoft Visual Studio

Procedure:

The way to write arithmetic expression is known as a notation. An arithmetic expression can be written in three different but equivalent notations, i.e., without changing the essence or output of an expression.

These notations are :

- Infix Notation
- Prefix (Polish) Notation
- Postfix (Reverse-Polish) Notation.

These notations are named as how they use operator in expression.

Infix Notation:

Infix notation is a method of writing mathematical expressions in which the operators are written between the operands. In other words, the operator appears in the middle of the two operands it is operating on.

For example, the infix notation for the expression "2 + 3" is "2 + 3". Infix notation is the most commonly used notation for writing mathematical expressions in everyday language. However, it is not always the most convenient notation for computer programs to evaluate, which is why other notations such as postfix and prefix are used in some programming languages.

Prefix Notation:

In this notation, operator is prefixed to operands, i.e. operator is written ahead of operands. For example, +ab. This is equivalent to its infix notation $a + b$. Prefix notation is also known as Polish Notation.

Postfix Notation:

This notation style is known as Reversed Polish Notation. In this notation style, the operator is postfix to the operands i.e., the operator is written after the operands. For example, ab+. This is equivalent to its infix notation $a + b$.

The following table briefly tries to show the difference in all three notations

Sr.No.	Infix Notation	Prefix Notation	Postfix Notation
1	$a + b$	$+ a b$	$a b +$
2	$(a + b) * c$	$* + a b c$	$a b + c *$
3	$a * (b + c)$	$* a + b c$	$a b c + *$
4	$a / b + c / d$	$+ / a b / c d$	$a b / c d / +$
5	$(a + b) * (c + d)$	$* + a b + c d$	$a b + c d + *$
6	$((a + b) * c) - d$	$- * + a b c d$	$a b + c * d -$

Infix to Postfix Conversion Algorithm:

1. Scan the Infix string from left to right.
2. Initialize an empty stack.
3. If the scanned character is an operand, add it to the Postfix string.
4. If the scanned character is an operator and if the stack is empty push the character to stack.
5. If the scanned character is an Operator and the stack is not empty, compare the precedence of the character with the element on top of the stack.
6. If top Stack has higher precedence over the scanned character pop the stack else push the scanned character to stack. Repeat this step until the stack is not empty and top Stack has precedence over the character.
7. Repeat 4 and 5 steps till all the characters are scanned.
8. After all characters are scanned, we have to add any character that the stack may have to the Postfix string.
9. If stack is not empty add top Stack to Postfix string and Pop the stack.
10. Repeat this step as long as stack is not empty.

Conversion to Postfix EXAMPLE:

$A + (B * C - (D / E - F) * G) * H$

Stack	Infix	Postfix
Empty	$A+(B*C-(D/E-F)*G)*H$	-
Empty	$+(B*C-(D/E-F)*G)*H$	A
+	$(B*C-(D/E-F)*G)*H$	A
+(	$B*C-(D/E-F)*G)*H$	A
+(	$*C-(D/E-F)*G)*H$	AB
+(*	$C-(D/E-F)*G)*H$	AB
+(*	$-(D/E-F)*G)*H$	ABC
+(	$(D/E-F)*G)*H$	ABC*
+(	$D/E-F)*G)*H$	ABC*
+(	$/E-F)*G)*H$	ABC*D
+(	$E-F)*G)*H$	ABC*D
+(	$-F)*G)*H$	ABC*DE
+(	$F)*G)*H$	ABC*DE/
+(	$F)*G)*H$	ABC*DE/
+(	$)G)*H$	ABC*DE/F
+(	$*G)*H$	ABC*DE/F-
+(	$G)*H$	ABC*DE/F-
+(	$)H$	ABC*DE/F-G

+	*H	ABC*DE/F-G*-
+	H	ABC*DE/F-G*-
+	End	ABC*DE/F-G*-H
Empty	End	ABC*DE/F-G*-H*+

Infix to Prefix Conversion Algorithm:

To convert an infix to prefix expression, we use the same algo for infix to postfix to convert Infix to Prefix.

Step 1:

Reverse the infix expression i.e $A+B*C$ will become $C*B+A$. Note while reversing each '(' will become ')' and each ')' becomes '('.

Step 2:

Obtain the postfix expression of the modified expression i.e $CB*A+$.

Step 3:

Infix to prefix conversion:

Expression = (A + B ^ C) * D + E ^ 5

Step 1:

Reverse the infix expression. 5 ^ E + D *) C ^ B + A (

Step 2:

Make Every '(' as ')' and every ')' as '(' 5 ^ E + D * (C ^ B + A)

Step 3:

Convert expression to postfix form.

Step 2 Expression	Stack	Postfix	Comment
5^E+D*(C^B+A)	Empty	-	Initial
^E+D*(C^B+A)	Empty	5	Print
E+D*(C^B+A)	^	5	Push
+D*(C^B+A)	^	5E	Push
D*(C^B+A)	+	5E^	Pop And Push
*(C^B+A)	+	5E^D	Print

(C^B+A)	+	5E^D	Push
C^B+A)	+(	5E^D	Push
^B+A)	+(	5E^DC	Print
B+A)	+(^	5E^DC	Push
+A)	+(^	5E^DCB	Print
A)	+(+	5E^DCB^	Pop And Push
)	+(+	5E^DCB^A	Print
End	+	5E^DCB^A+	Pop Until '('
End	Empty	5E^DCB^A+*+	Pop Every element

Step 4:

Reverse the expression. + * + A ^ B C D ^ E 5

Algorithm for Prefix to Postfix:

1. Read the Prefix expression in reverse order (from right to left).
2. If the symbol is an operand, then push it onto the Stack.
3. If the symbol is an operator, then pop two operands from the Stack .
 - 3.1 Create a string by concatenating the two operands and the operator after them.
 - 3.2 string = operand1 + operand2 + operator.

- 3.3 And push the resultant string back to Stack.
4. Repeat the above steps until end of Prefix expression.

Tasks

Task 1:

Write a C++ program to convert an Infix expression into Prefix expression. Also check whether Infix expression is mathematically correct or not.

CODE:

OUTPUT:

```
Microsoft Visual Studio Debug Console
Enter the infix expression: ( A + B ^ C ) * D + E ^ 5
The infix expression is not mathematically correct.

E:\dev c++\LAB REPORT 7\Debug\LAB REPORT 7.exe (process 12328) exited with code 0.
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the console when debugging stops.
Press any key to close this window . . .
```

```
E:\HUZAIFA SHAHBAZ 21-CE-009\infixtopostfixexpression.exe
Enter infix expression: (A+B^C)*D+E^5
Prefix expression: ++A^BCD^E5

-----
Process exited after 38.56 seconds with return value 0
Press any key to continue . . .
```

Task 2:

Write a C++ program to convert an Infix expression into Postfix expression.

CODE:

```

1  // Infix to Postfix Converter
2  #include <iostream>
3  using namespace std;
4  class Node
5  {
6  public:
7      char data;
8      Node* next;
9  };
10 class Stack
11 {
12 public:
13     Node* top;
14     Stack()
15     {
16         top = NULL;
17     }
18     void push(char c)
19     {
20         Node* temp = new Node();
21         temp->data = c;
22         temp->next = top;
23         top = temp;
24     }
25     char pop()
26     {
27         if (top == NULL)
28         {
29             return '\0';
30         }
31         Node* temp = top;
32         top = top->next;
33         char popped = temp->data;
34         delete temp;
35         return popped;
36     }
37     char peek()
38     {
39         if (top == NULL)
40         {
41             return '\0';
42         }
43         return top->data;
44     }
45 };
46 bool isOperator(char c)
47 {
48     if (c == '+' || c == '-' || c == '*' || c == '/' || c == '(' || c == ')') {
49         return true;
50     }
51     return false;
52 }
53 int precedence(char c)
54 {
55     if (c == '(')
56     {
57         return 1;
58     }
59     else if (c == '+' || c == '-')
60     {
59         return 2;
60     }
61     else if (c == '*' || c == '/')
62     {
63         return 3;
64     }
65     else if (c == '(' || c == ')')
66     {
67         return -1;
68     }
69 }
70 string infixToPostfix(string infix)
71 {
72     Stack stack;
73     string postfix = "";
74     for (int i = 0; i < infix.length(); i++)
75     {
76         char c = infix[i];
77         if (isalpha(c) || isdigit(c))
78         {
79             postfix += c;
80         }
81         else if (c == '(')
82         {
83             stack.push(c);
84         }
85         else if (c == ')')
86         {
87             while (stack.peek() != '(')
88             {
89                 postfix += stack.pop();
90             }
91             stack.pop();
92         }
93         else
94         {
95             while ((stack.top != NULL && precedence(c) <= precedence(stack.peek())) {
96                 postfix += stack.pop();
97             }
98             stack.push(c);
99         }
100     }
101     while (stack.top != NULL)
102     {
103         postfix += stack.pop();
104     }
105     return postfix;
106 }
107 int main()
108 {
109     string infix = "a+b*c-(d/e+f)*g";
110     string postfix = infixToPostfix(infix);
111     cout << "Infix Expression: " << infix << endl;
112     cout << "Postfix Expression: " << postfix << endl;
113     return 0;
114 }

```

OUTPUT:

```

Select Microsoft Visual Studio Debug Console
Infix Expression: a+b*c-(d/e+f)*g
Postfix Expression: abc*+de/f+g*-

E:\dev c++\LAB RPEORT 7 PROGRAMS 3\64\Debug\LAB RPEORT 7 PROGRAMS 3.exe (process 10444) exited with code 0.
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the console when debugging stops.
Press any key to close this window . . .

```

```
E:\HUZAIFA SHAHBAZ 21-CE-009\infixtoprefixexpression.exe
Enter infix expression: A+(B*C-(D/E-F)*G)*H
Postfix expression: +A*-*BC*-/DEFGH
-----
Process exited after 54.15 seconds with return value 0
Press any key to continue . . .
```

Task 3:

Write a C++ program to convert a Prefix expression into Postfix expression.

CODE:

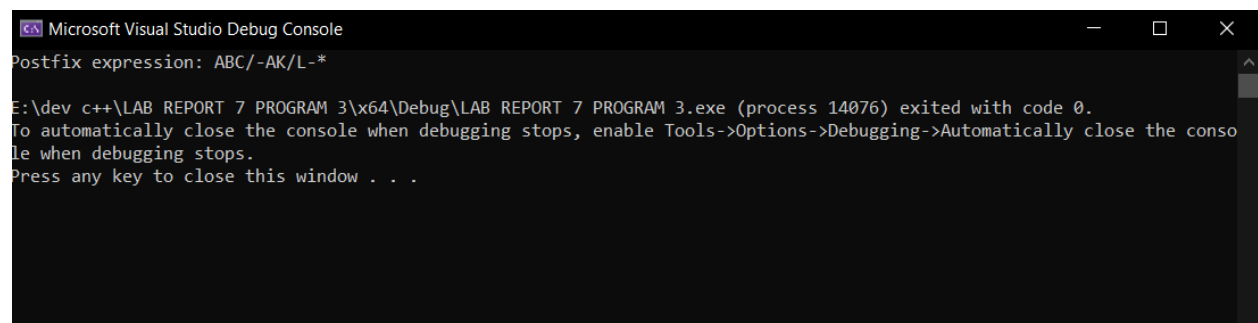
```
1  #include <iostream>
2  #include <stack>
3  using namespace std;
4  bool isOperand(char ch) {
5      return (ch >= 'a' && ch <= 'z') || (ch >= 'A' && ch <= 'Z');
6  }
7  string prefixToPostfix(string prefix) {
8      stack<string> s;
9      for (int i = prefix.length() - 1; i >= 0; i--)
10     {
11         if (isOperand(prefix[i])) {
12             string operand1, prefix[i];
13             s.push(operand1);
14         }
15         else
16         {
17             string operand1 = s.top();
18             s.pop();
19             string operand2 = s.top();
20             s.pop();
21             string result = operand1 + operand2 + prefix[i];
22             s.push(result);
23         }
24     }
25     return s.top();
26 }
27 int main()
28 {
29     string prefix = "-A/BC-/AKL";
30     string postfix = prefixToPostfix(prefix);
31     cout << "Postfix expression: " << postfix << endl;
32     return 0;
33 }
```

OUTPUT:



```
E:\HUZAIFA SHAHBAZ 21-CE-009\prefixtopostfixconversion.exe
Prefix expression: *-A/BC-/AKL
Postfix expression: ABC/-AK/L-*

-----
Process exited after 0.07601 seconds with return value 0
Press any key to continue . . .
```



```
Microsoft Visual Studio Debug Console
Postfix expression: ABC/-AK/L-*

E:\dev c++\LAB REPORT 7 PROGRAM 3\Debug\LAB REPORT 7 PROGRAM 3.exe (process 14076) exited with code 0.
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the console when debugging stops.
Press any key to close this window . . .
```

Conclusion:

Each notation has its own advantages and disadvantages. **Infix notation** is more natural to read and write for humans, but it requires the use of parentheses to disambiguate expressions. **Prefix and postfix notations** eliminate the need for parentheses, but they can be more difficult for humans to read and write. However, prefix and postfix notations are easier for computers to evaluate because they can be evaluated using a stack data structure.

